



Онлайн образование

- otus.ru

 Проверить, идет ли запись

Меня хорошо видно && слышно?



Тема вебинара

Kafka



Заигрин Вадим

Ведущий эксперт по технологиям, Сбербанк

vzaigrin@yandex.ru

<https://t.me/vzaigrin>



Преподаватель



Вадим Заигрин

Более 30 лет в ИТ:

- Big Data
 - Data Engineer
 - Data Science
- Разработка
 - Scala, Java, Python, C, Lisp
- IT Infrastructure
 - Администрирование
 - Сопровождение
 - Архитектура

Big Data проекты в банках, телекоме и в рознице.



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в Telegram



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Карта курса

Введение

Хранение данных

Загрузка данных

Запросы к данным и
трансформация

Предоставление данных для
аналитики и машинного
обучения

Базы данных NoSQL

Выпускной проект

Маршрут вебинара



Цели вебинара

- | | |
|----|--|
| 1. | Познакомимся с потоковой обработкой данных |
| 2. | Узнаем что такое Kafka и зачем она нужна |
| 3. | Рассмотрим работу с Kafka |

Смысл

- | | |
|----|---|
| 1. | Узнаем для чего использовать потоковую обработку данных |
| 2. | Сможем начать работать с Kafka |

Поток данных

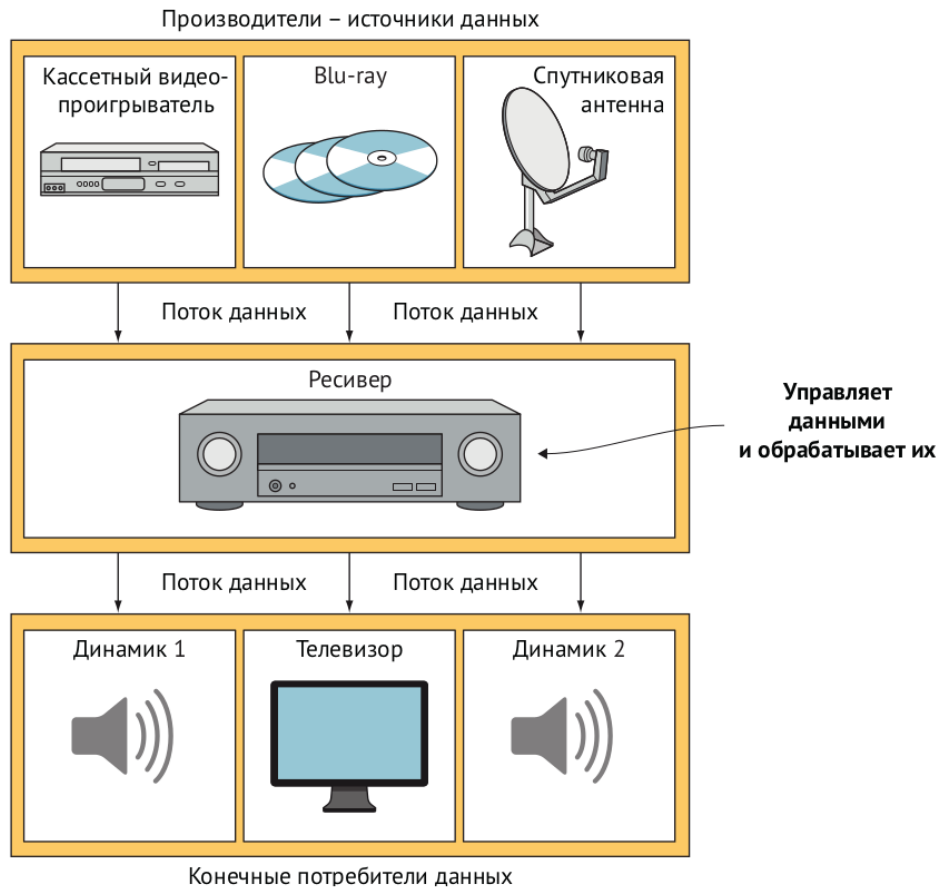
Поток данных

- **Поток данных** (stream) — это именованный набор сообщений.
- **Поток** — неограниченный набор записей
- **Пакет** — конечный набор записей

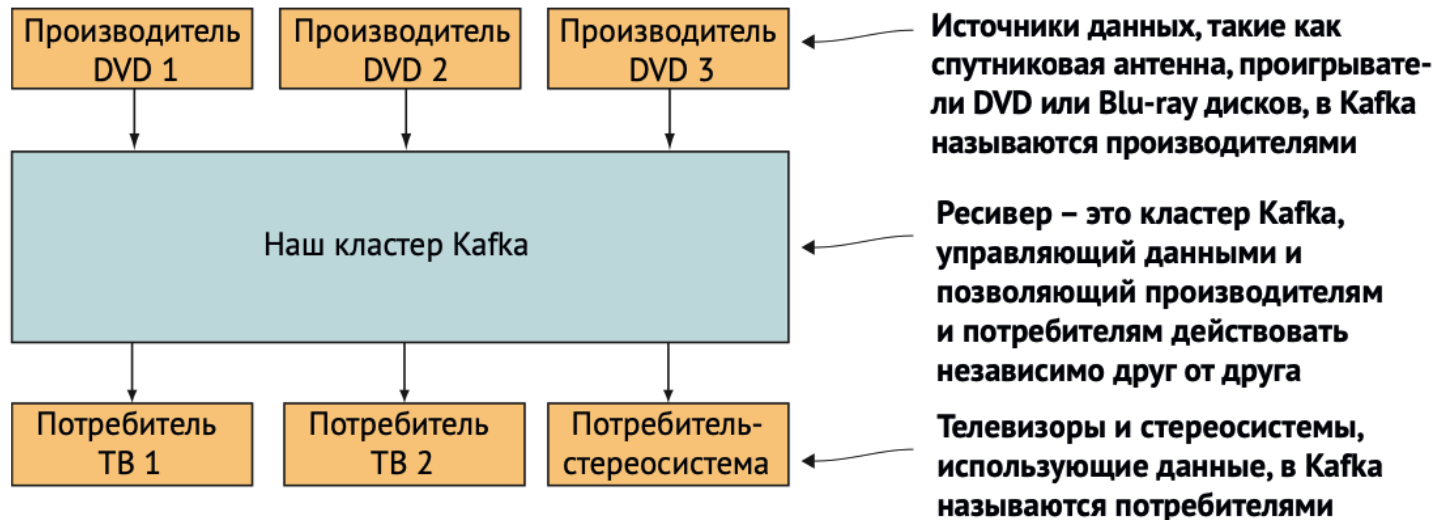
Потребности в потоковой обработки

- Ориентация на *сейчас*
 - Финансовые транзакции
 - Действия в онлайн-магазинах
 - Перемещения пользователей
 - Социальные сети
 - Промышленные датчики
- Результат нужен практически в режиме реального времени

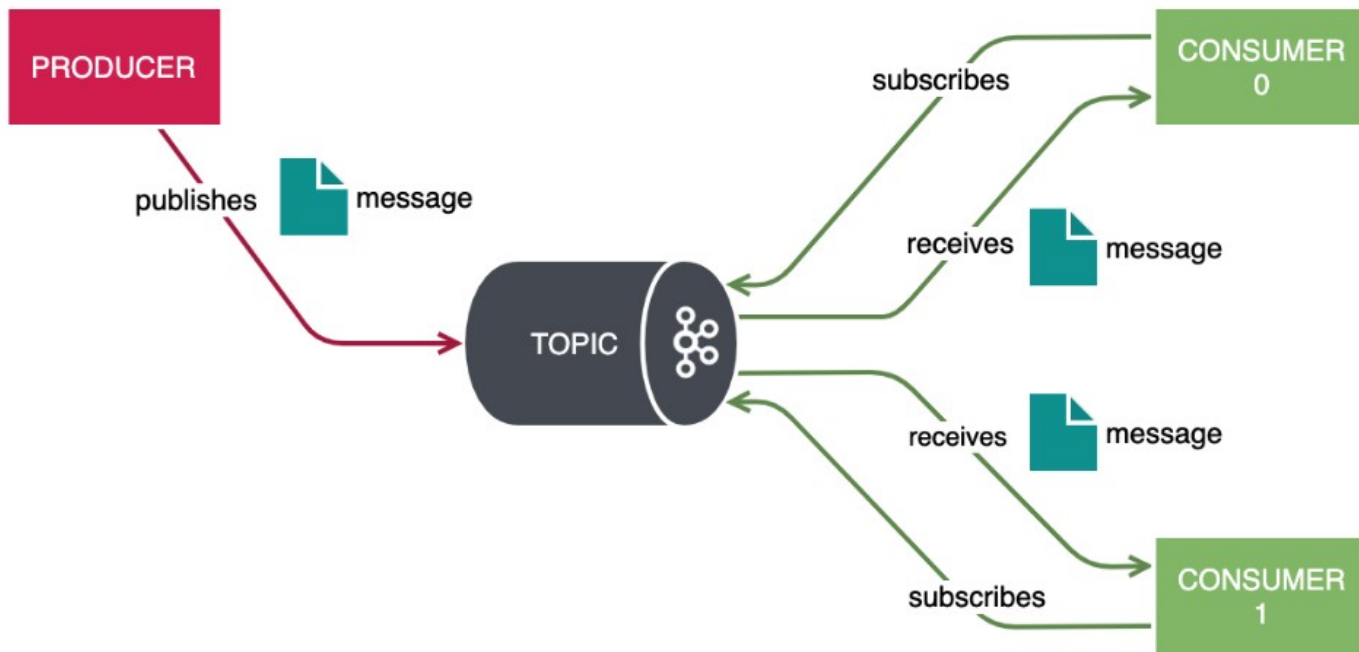
Потоки в реальной жизни



Место Kafka в потоке данных

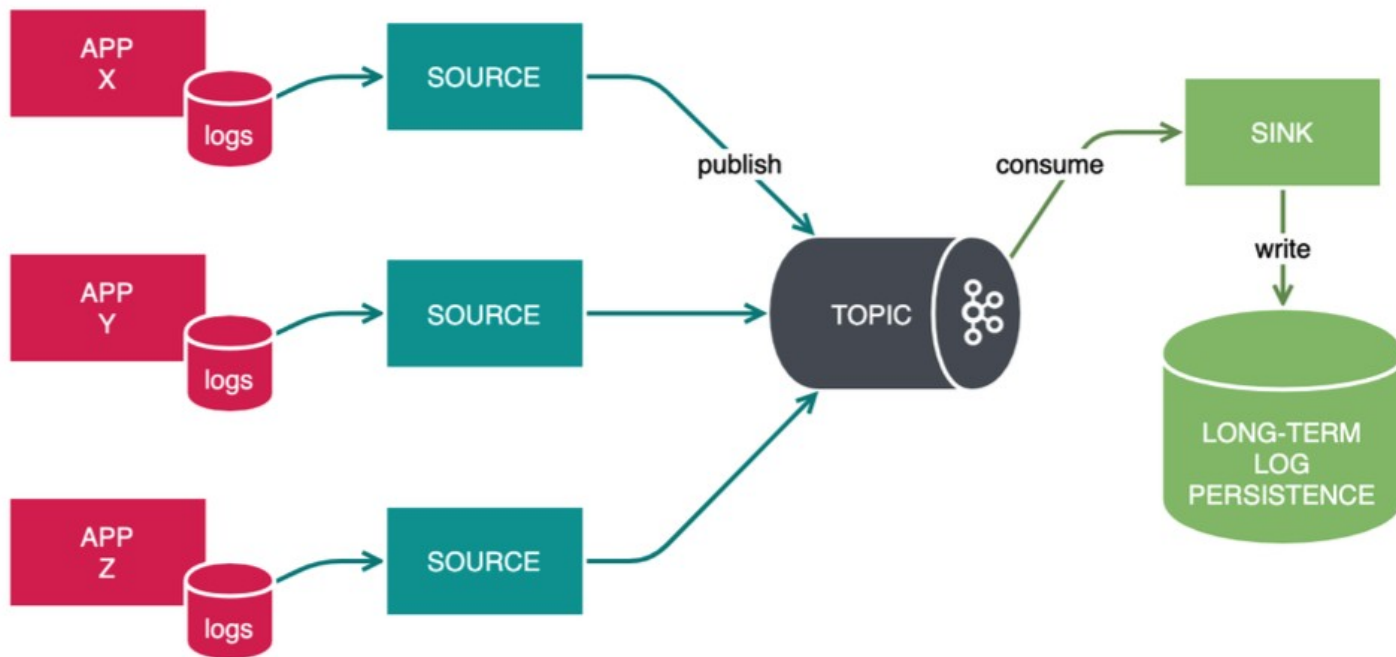


Publish / Subscribe

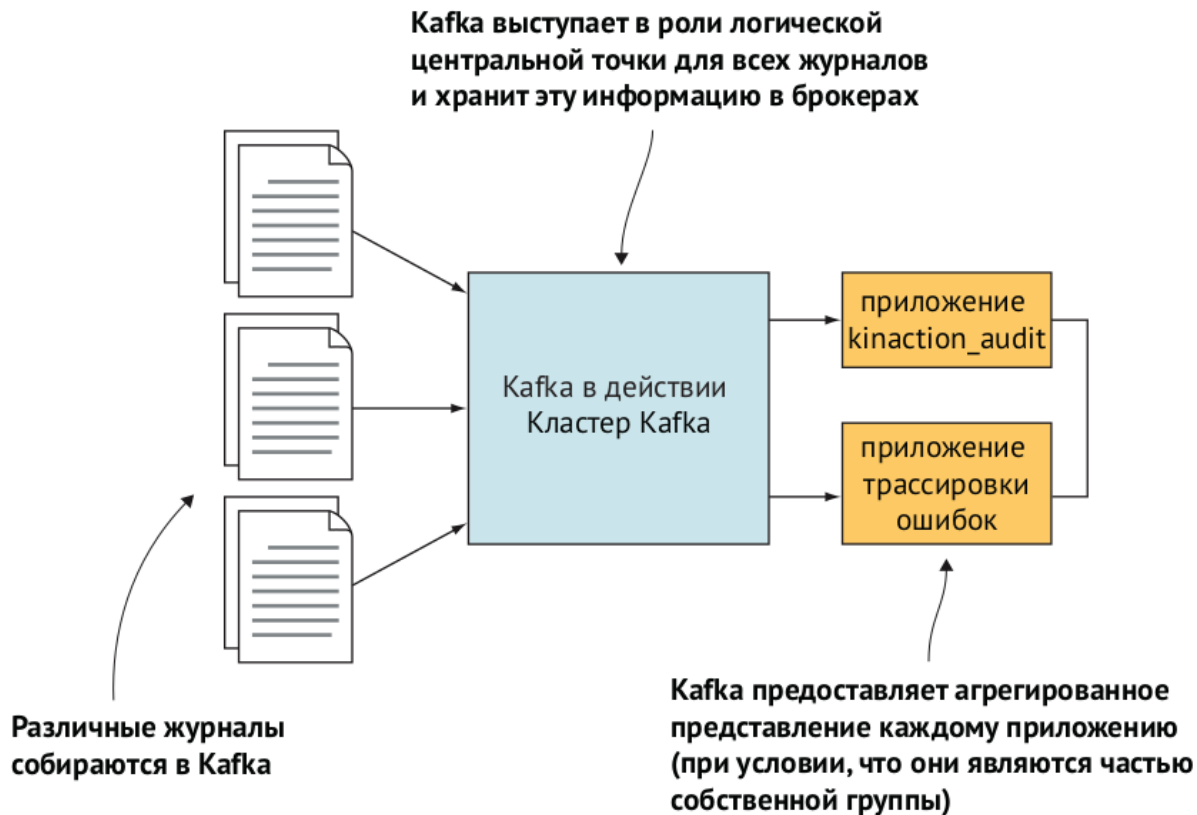


Примеры использования

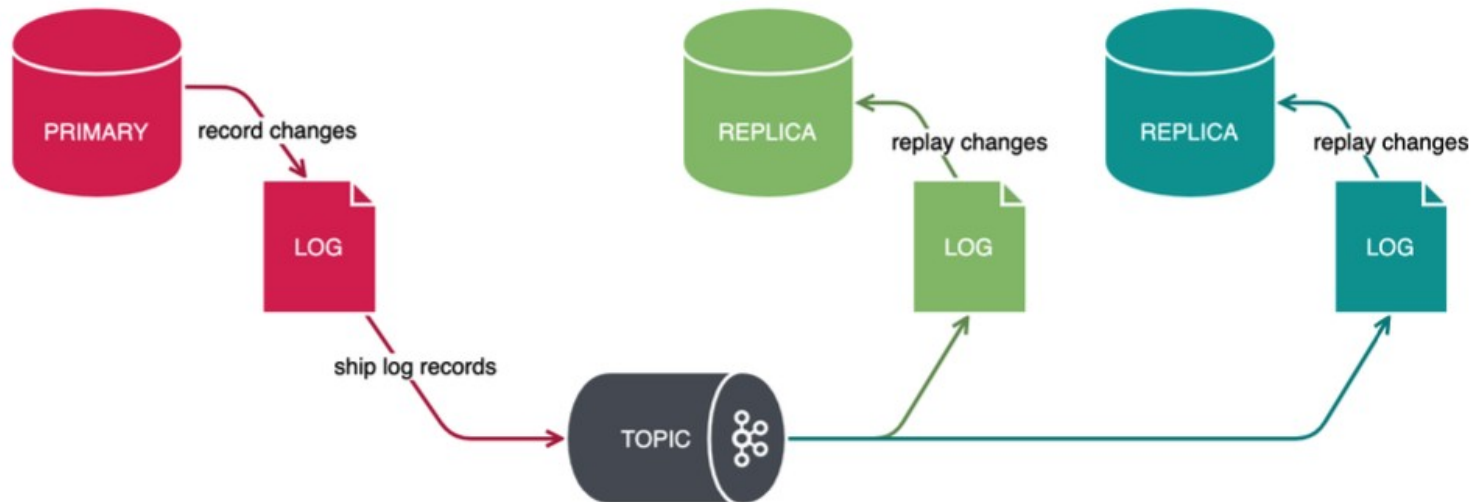
Агрегация журналов



Обработка журналов



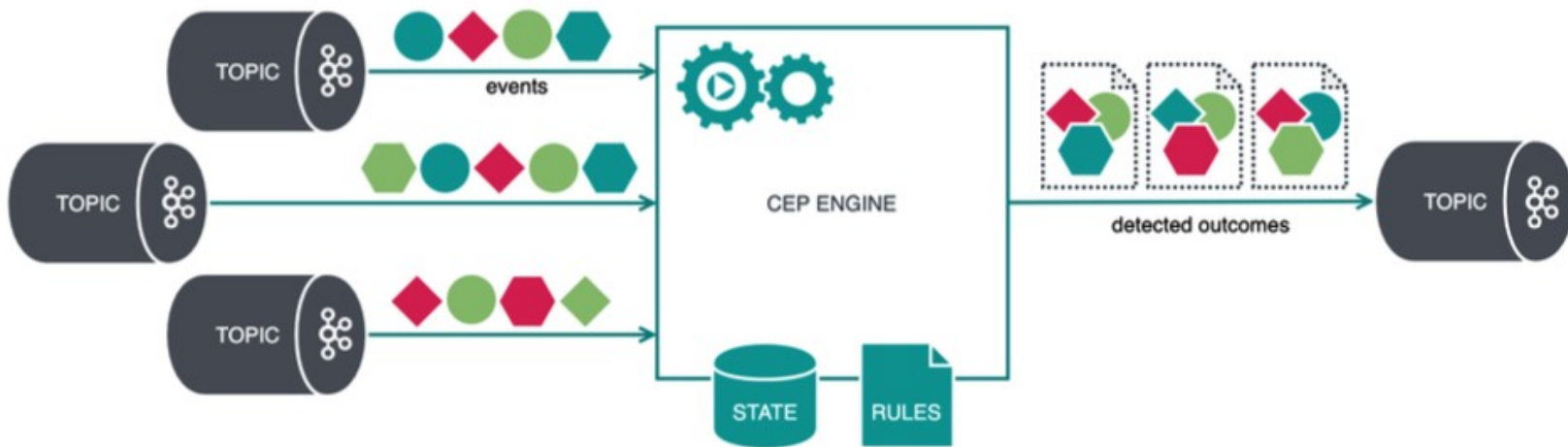
Доставка журналов (log shipping)



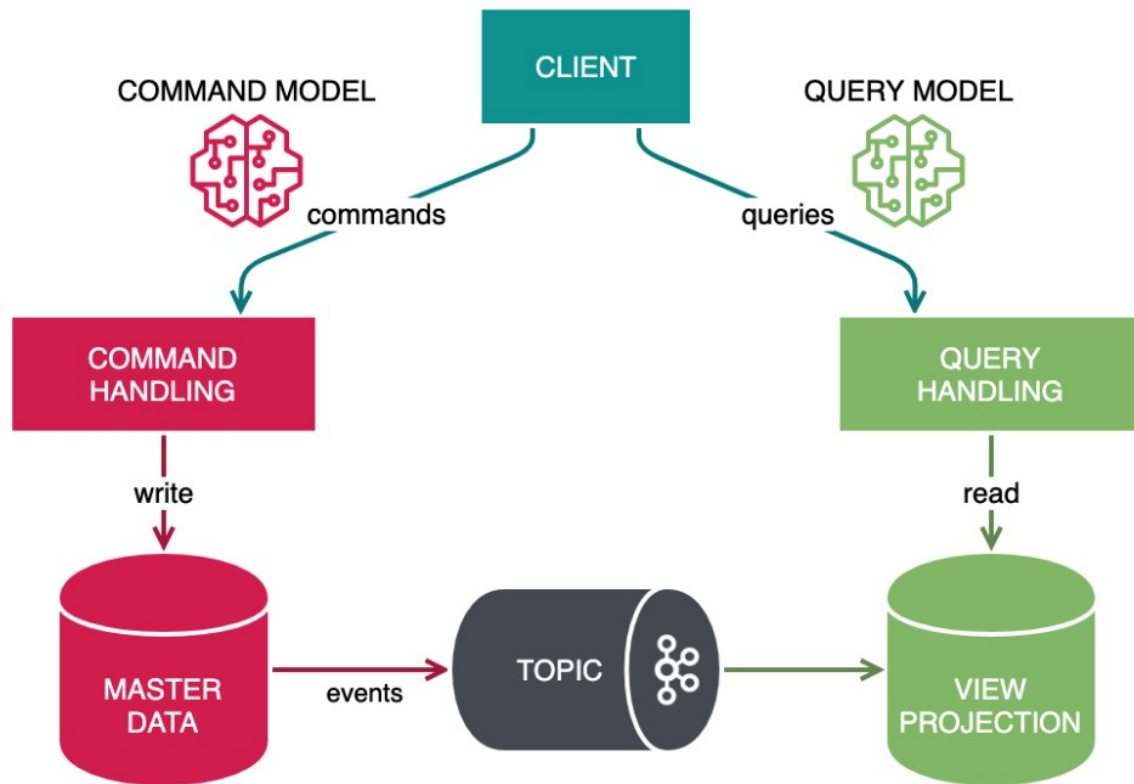
Интернет вещей



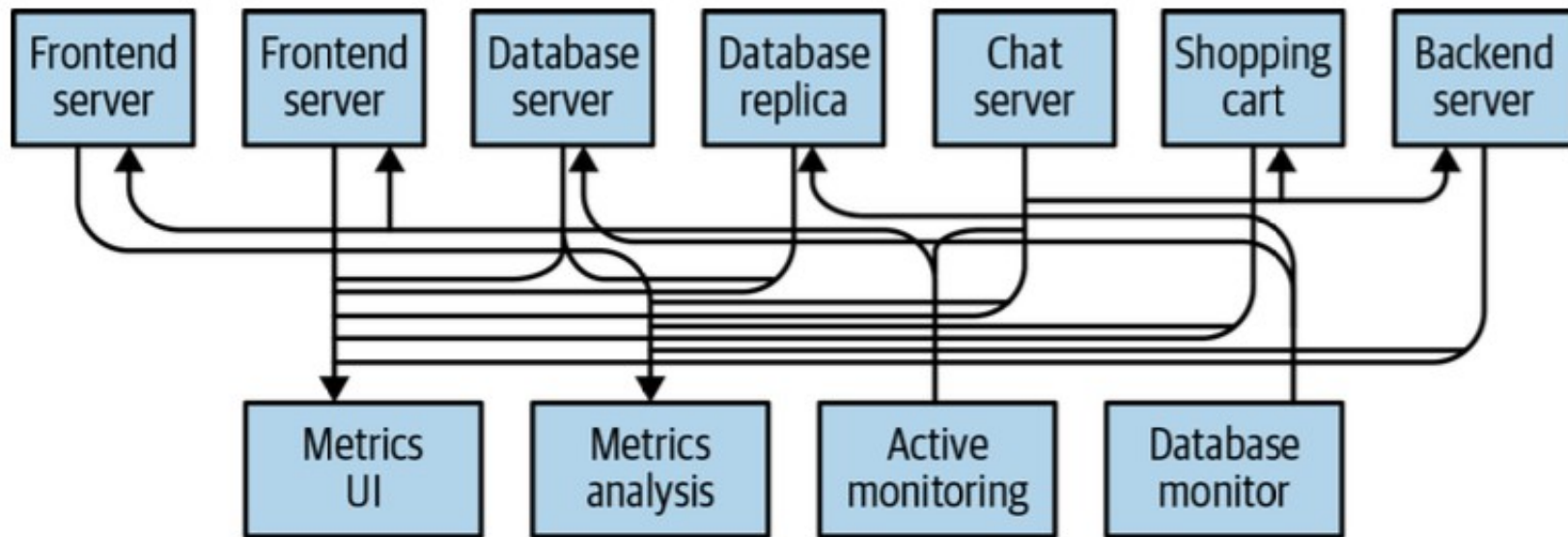
Сложная обработка событий (Complex Event Processing)



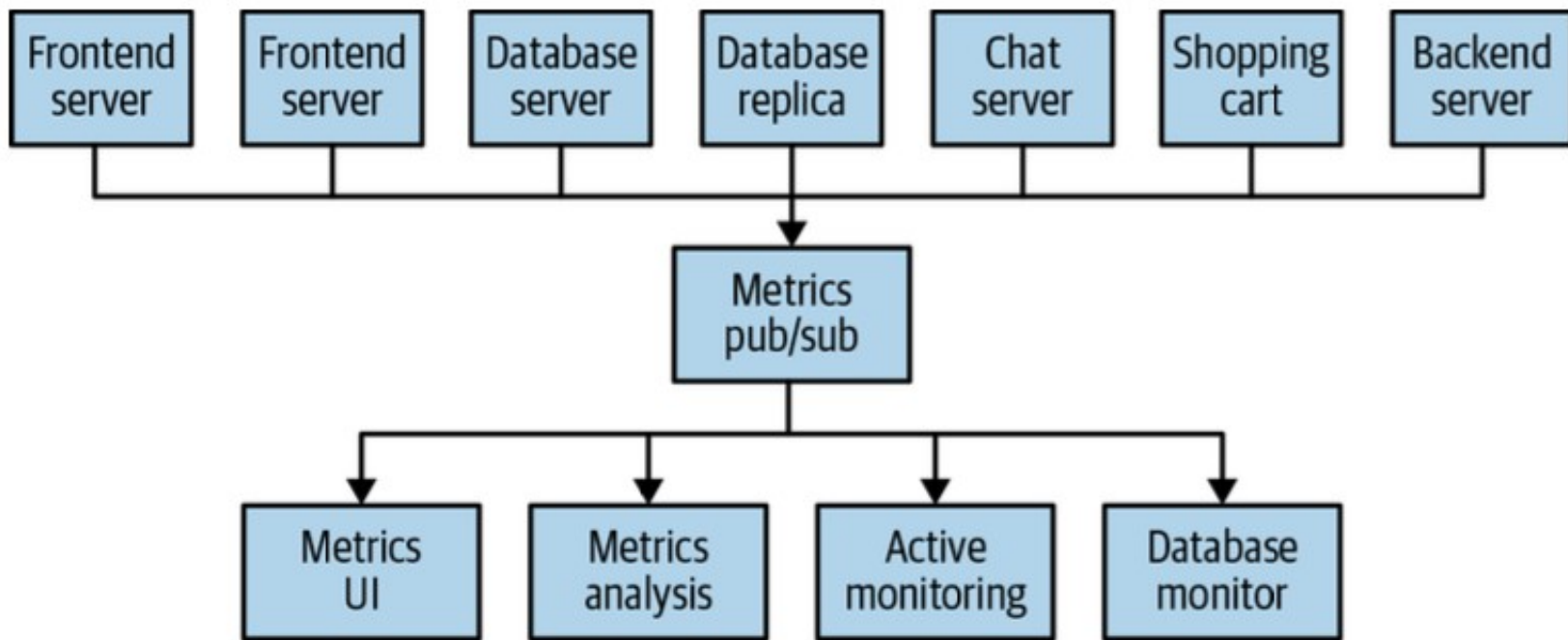
Command-Query Responsibility Segregation (CQRS)



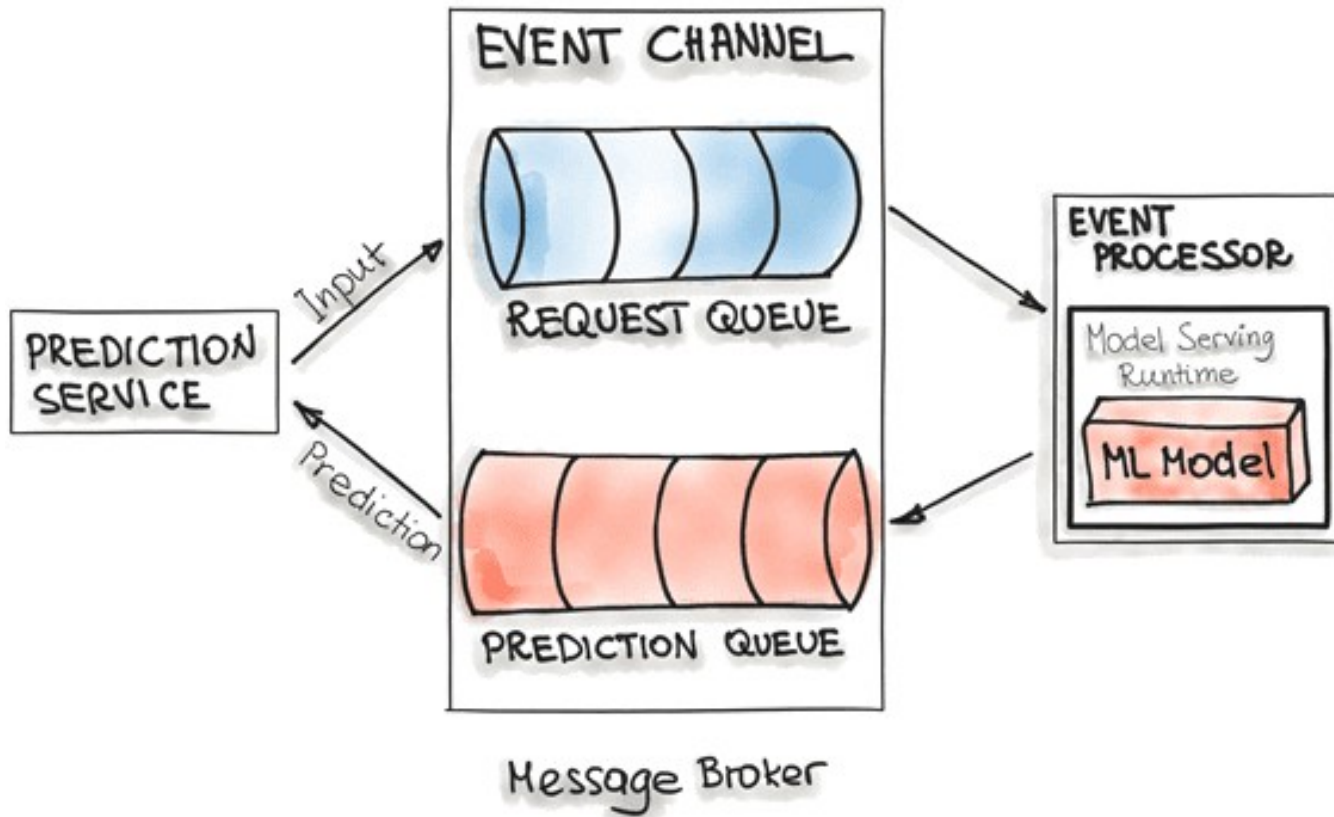
Пример традиционной архитектуры



Архитектура, ориентированная на события



Model-on-Demand



Знакомство с Kafka

Что такое Kafka?

Kafka – это распределенная потоковая платформа, которая обладает тремя основными возможностями:

- публиковать записи и подписываться на очереди сообщений
- хранить записи с отказоустойчивостью
- обрабатывать потоки по мере их возникновения

Платформа:

- API:
 - Producer
 - Consumer
 - Admin
- Kafka Streams
- Kafka Connect
- ksqlDB

Основные концепции

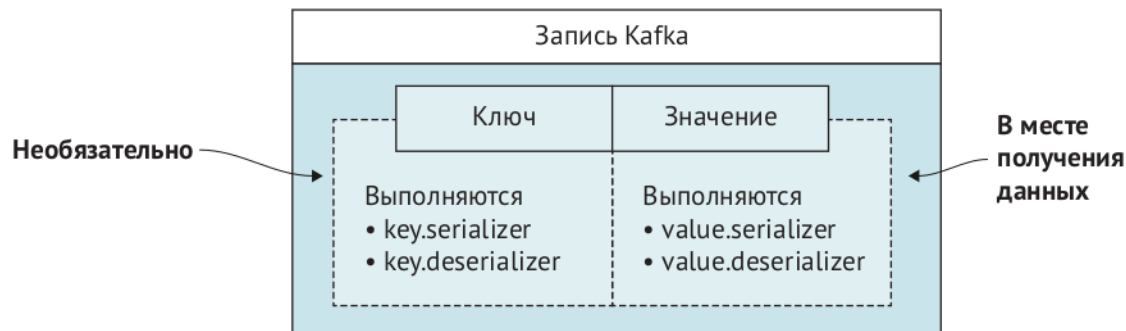
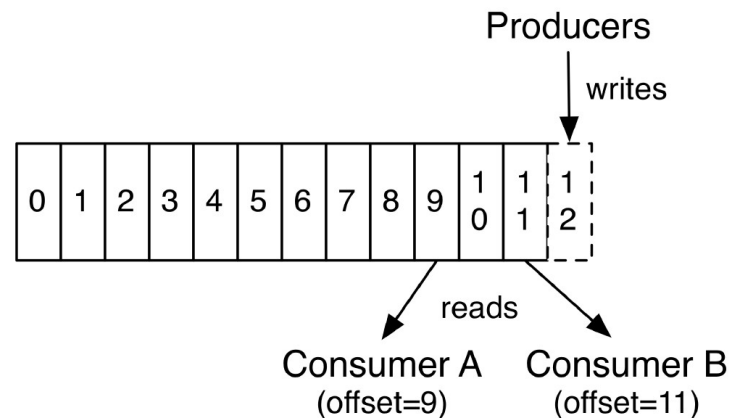
Record – элемент данных типа ключ-значение

Topic – имя потока с данными

Offset – позиция записи

Producer – процесс, публикующий записи

Consumer – процесс, читающий записи

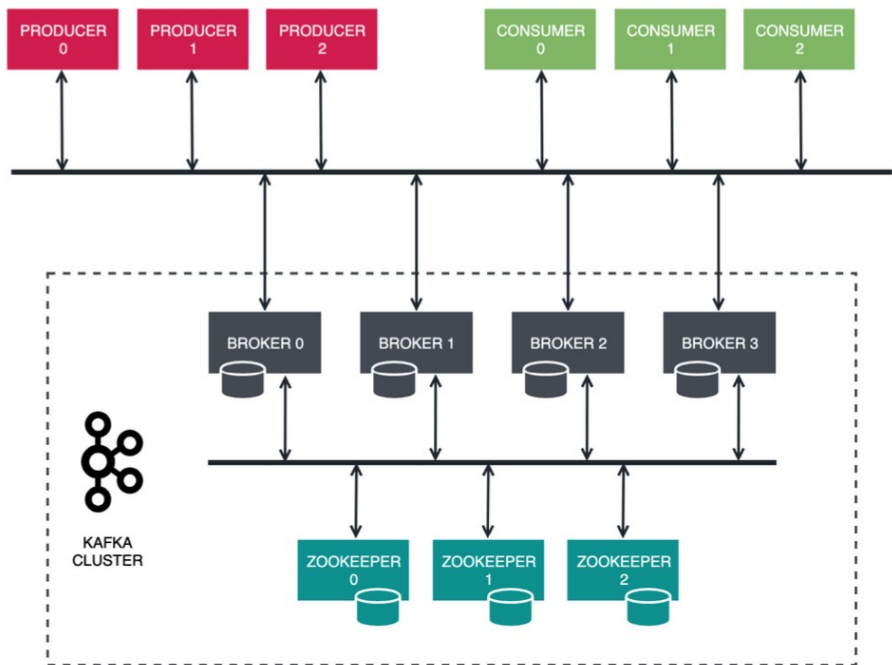


Архитектура Kafka

Broker – управление данными, взаимодействие с клиентами

Zookeeper – членство брокеров в кластере, выборы контроллера

Контроллер – это брокер, который отвечает за выбор ведущих реплик для разделов



Запускаем Kafka

Запускаем Kafka

1) Скачиваем и разворачиваем

```
$ tar -xzf kafka_2.13-3.4.0.tgz
```

```
$ cd kafka_2.13-3.4.0
```

2) Запускаем сервисы

```
$ bin/zookeeper-server-start.sh -daemon config/zookeeper.properties
```

```
$ bin/kafka-server-start.sh -daemon config/server.properties
```

3) Создаём тему

```
$ bin/kafka-topics.sh --create --topic quickstart --bootstrap-server localhost:9092
```

```
$ bin/kafka-topics.sh --describe --topic quickstart --bootstrap-server localhost:9092
```

4) Запишем что-нибудь в тему

```
$ bin/kafka-console-producer.sh --topic quickstart --bootstrap-server localhost:9092
```

```
This is my first event
```

```
This is my second event
```

5) Прочитаем записи

```
$ bin/kafka-console-consumer.sh --topic quickstart --from-beginning ---bootstrap-server localhost:9092
```

```
This is my first event
```

```
This is my second event
```

Kafka в Docker

version: '2.1'

services:

zookeeper:

image: confluentinc/cp-zookeeper:latest

ports:

- "2181:2181"

environment:

ZOOKEEPER_CLIENT_PORT: 2181

ZOOKEEPER_TICK_TIME: 2000

kafka:

image: confluentinc/cp-kafka:latest

ports:

- "9092:9092"

environment:

KAFKA_BROKER_ID: 1

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_ADVERTISED_LISTENERS: LISTENER_DOCKER_INTERNAL://kafka:19092, LISTENER_DOCKER_EXTERNAL://\${DOCKER_HOST_IP:-127.0.0.1}:9092

KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: LISTENER_DOCKER_INTERNAL:PLAINTEXT, LISTENER_DOCKER_EXTERNAL:PLAINTEXT

KAFKA_INTER_BROKER_LISTENER_NAME: LISTENER_DOCKER_INTERNAL

KAFKA_LOG4J_ROOT_LOGLEVEL: INFO

KAFKA_CONFLUENT_SUPPORT_METRICS_ENABLE: "false"

depends_on:

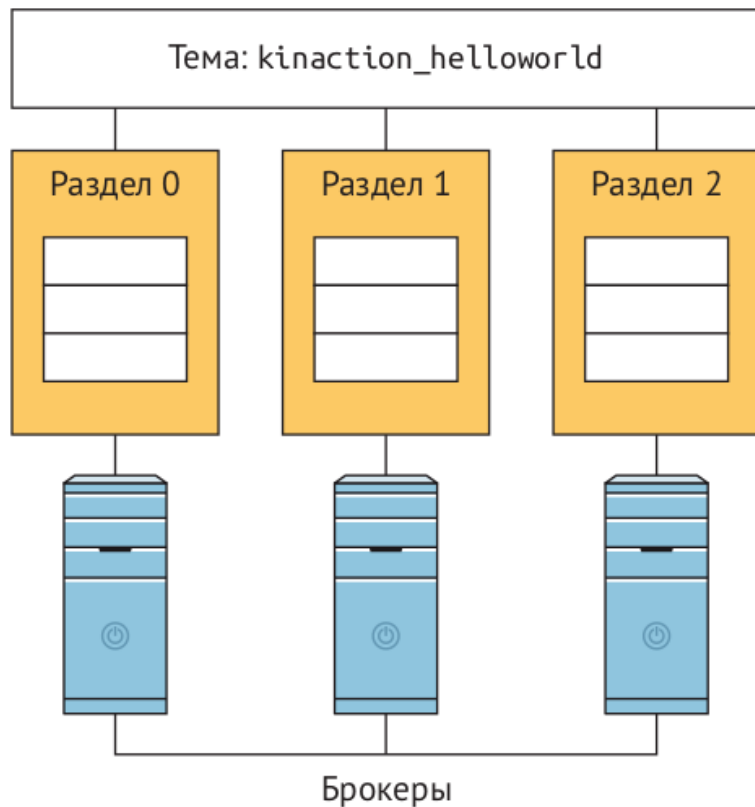
- zookeeper

Основные операции

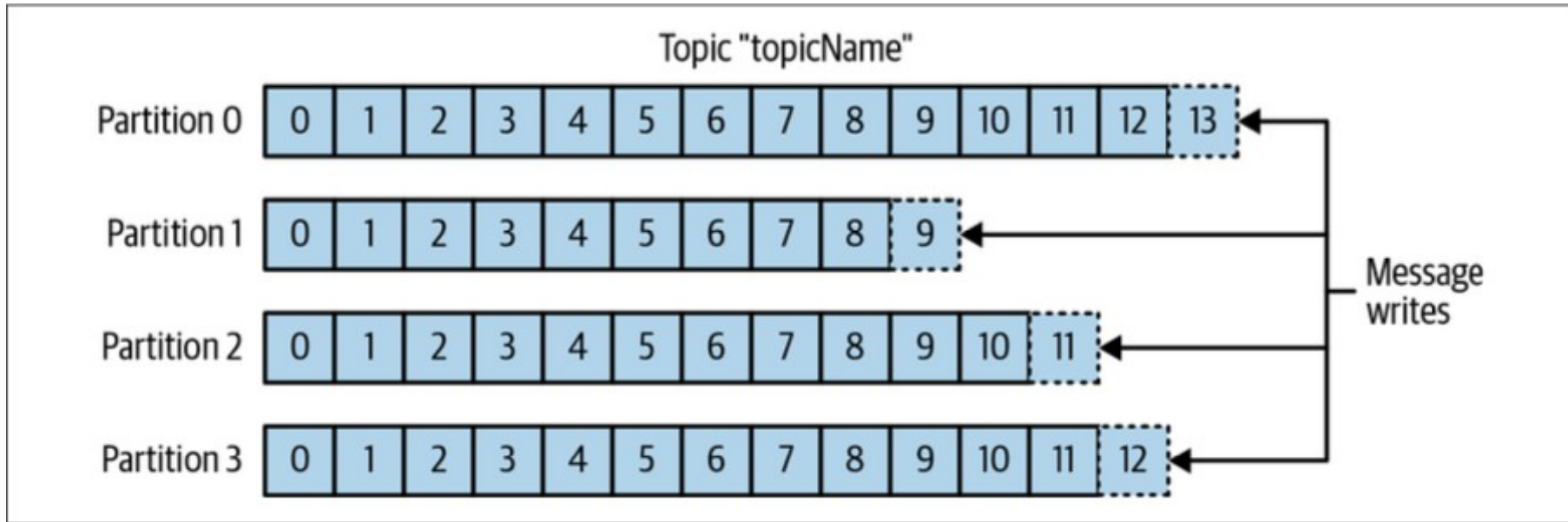
- `zookeeper-server-start.sh` — запуск Zookeeper
- `zookeeper-server-stop.sh` — останов Zookeeper
- `kafka-server-start.sh` — запуск Kafka брокера
- `kafka-server-stop.sh` — останов Kafka брокера
- `kafka-console-producer.sh` — консольный Producer
- `kafka-console-consumer.sh` — консольный Consumer
- `kafka-topics.sh` — работа с темами (создать, удалить, изменить, посмотреть)

Темы и разделы

Темы и разделы (секции, partitions)



Запись в тему с разделами



Разделы группируют данные по ключу

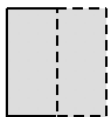
Входящие сообщения:

{foo, данные сообщения}
{bar, данные сообщения}

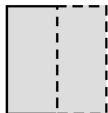
Секция, в которую будет отправлено сообщение, определяется его ключами. Эти ключи не пусты.

Байтовое представление ключа используется для вычисления хеша

Секция 0 $\text{hashCode(fooBytes)} \% 2 = 0$



Секция 1 После определения секции сообщение добавляется в конец соответствующего журнала



$\text{hashCode(barBytes)} \% 2 = 1$

Функционирование журналов

- `log.dir` – место хранения журналов
- Подкаталог – тема
- Кол-во подкаталогов – кол-во разделов
- Формат названия: имя-темы_номер-раздела

`/logs`

`/logs/topicA_0`

В topicA — одна секция

`/logs/topicB_0`

В topicB — три секции

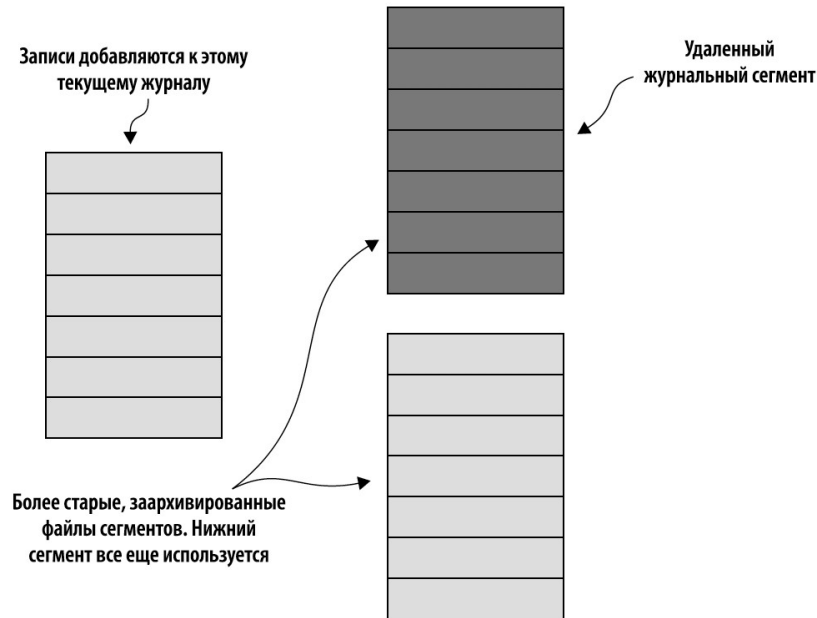
`/logs/topicB_1`

`/logs/topicB_2`

Удаление журналов

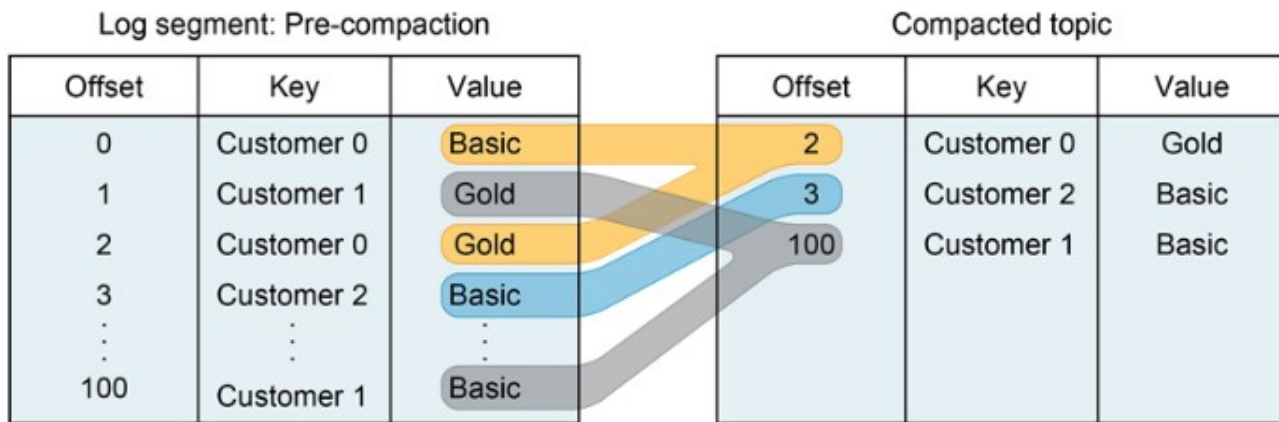
Двухэтапная стратегий удаления:

- архивация журналов в сегменты (timestamp + log.roll.ms)
- удаление старых сегментов



Сжатие журналов

- Kafka поддерживает удаление старых записей (сжатие) за счёт стратегии хранения:
 - `delete` – удаление событий, чей возраст превышает срок хранения
 - `compact` – сохранение только последних значений для каждого ключа
- Сжатие возможно только для тем с непустыми ключами



Распределённый журнал

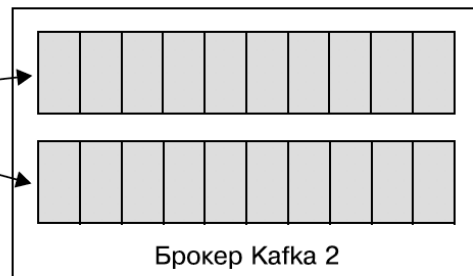
Данный топик состоит из шести секций, расположенных в трехузловом кластере Kafka. Журнал топика распределен по трем узлам. Здесь показаны только ведущие брокеры для секций топика

Генератор



Секция 0

Секция 1



Секция 2

Секция 3



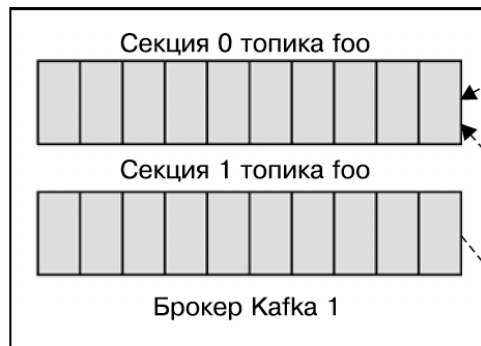
Секция 4

Секция 5

Если ключей в сообщении нет, секции выбираются циклическим образом. В противном случае они выбираются посредством деления хеша ключа по модулю количества секций

Репликация

Топик foo состоит из двух секций, его уровень репликации — 3. Штриховые линии между секциями указывают на ведущие брокеры данных секций. Генераторы записывают данные на ведущие брокеры, а ведомые брокеры читают данные с них



Брокер 1 — ведущий для секции 0 и ведомый для секции 1 на брокере 3



Брокер 2 — ведомый как для секции 0 на брокере 1, так и для секции 1 на брокере 3

Брокер 3 — ведомый для секции 0 на брокере 1 и ведущий для секции 1



Поведение при отказе брокера

Топик foo состоит из двух секций, его уровень репликации — 3. Изначально у него следующие ведущие и ведомые брокеры:
Брокер 1 — ведущий для секции 0 и ведомый для секции 1
Брокер 2 — ведомый для секции 0 и секции 1
Брокер 3 — ведомый для секции 0 и ведущий для секции 1

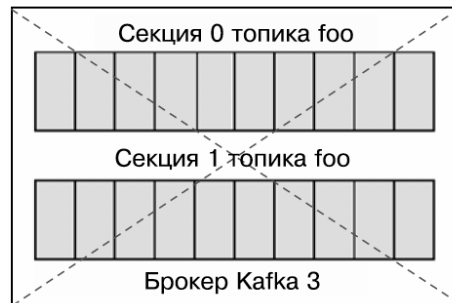
Брокер 3 перестал реагировать на контрольные сигналы ZooKeeper



Шаг 1: будучи ведущим, брокер 1 обнаружил сбой брокера 3

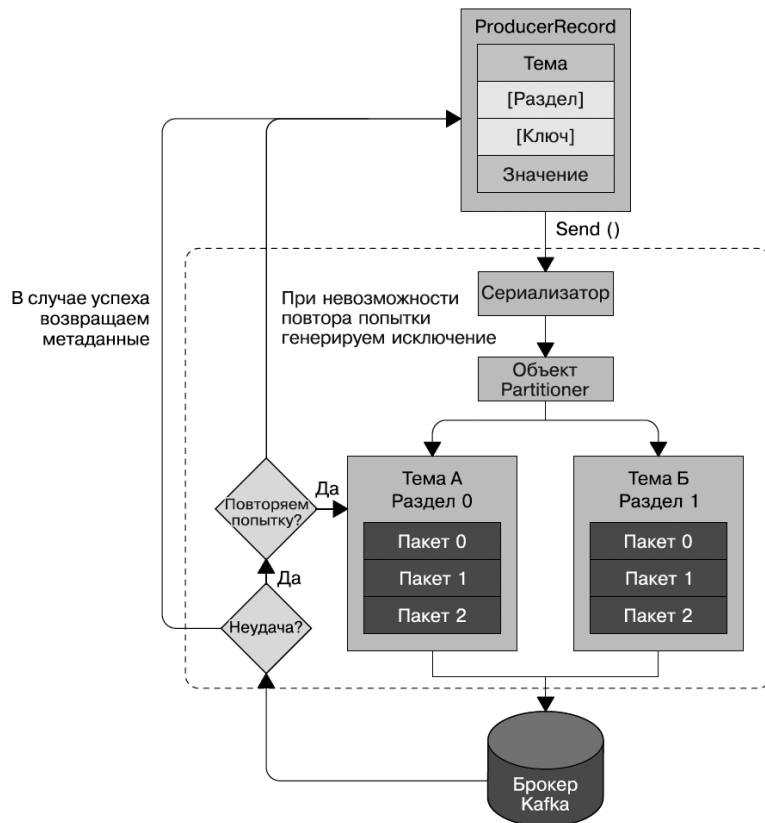


Шаг 2: контроллер делает ведущим для секции 1 брокер 2 вместо брокера 3. Все записи секции 1 теперь будут попадать на брокер 2, а брокер 1 будет потреблять сообщения для секции 1 с брокера 2



Producer

Producer – запись сообщений в Kafka



Свойства Producer

Обязательные свойства:

- `bootstrap.servers` – список пар `host:port` брокеров
- `key.serializer` – имя класса, используемого для сериализации ключей записей
- `value.serializer` – имя класса, используемого для сериализации значений записей

Необязательные свойства:

- `acks` – число получивших сообщение реплик секций, требующееся для признания производителем операции записи успешной.
 - `acks = 0` – Производитель не ждёт ответа от брокера, чтобы считать отправку успешной
 - `acks = 1` – Производитель получает ответ от брокера об успешном получении, как только ведущая реплика получит сообщение
 - `acks = all` – Производитель получает ответ от брокера об успешном получении, когда оно дойдёт до всех синхронизируемых реплик
- `retries` – число попыток отправить сообщение
- `partitioner.class` – класс для партиционирования

<https://kafka.apache.org/documentation/#producerconfigs>



Property acks=0

1. Производитель
передает сообщение
лидеру раздела

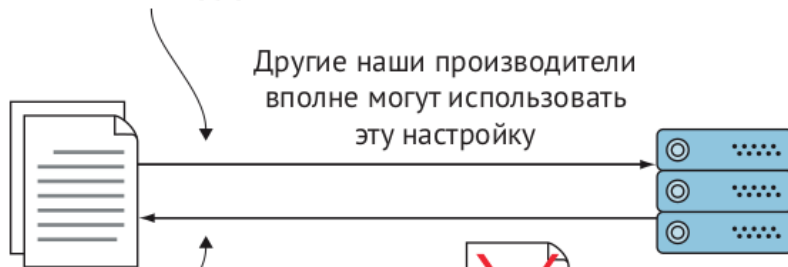
2. Лидер не ждет, чтобы
узнать, была ли запись
выполнена успешно

3. Мы не знаем, было ли сообщение успешно
записано лидером, поэтому мы не можем
знать о состоянии ведомых реплик и об
успехе или неудаче передачи сообщения ими



Property acks=1

1. Производитель передает сообщение лидеру раздела

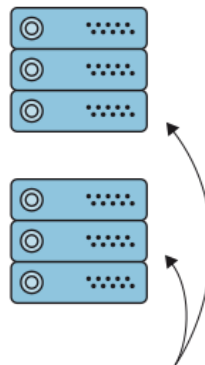


2. Лидер раздела отвечает, что сообщение было записано успешно

3. Лидер терпит сбой до того, как успеет отправить сообщение ведомым репликам. Это означает, что сообщение может быть потеряно для остальных брокеров

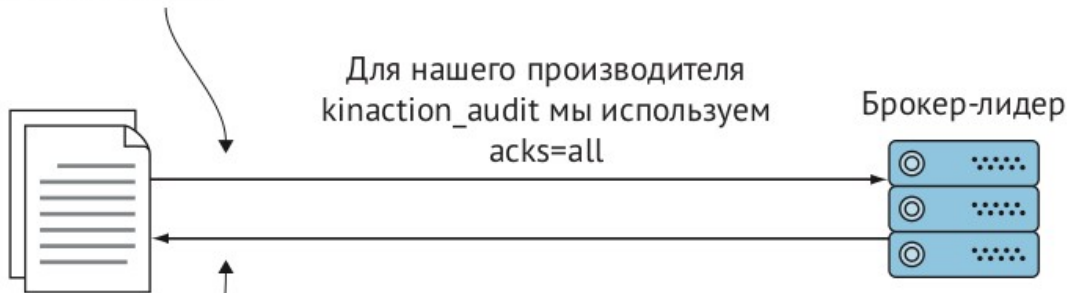


4. Эти брокеры так и не получают сообщение, даже если его получал лидер

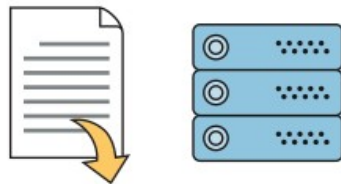


Property acks=all

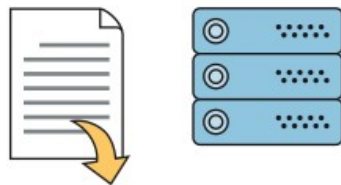
1. Производитель
передает сообщение
лидеру раздела



2. Лидер ждет, пока все
брокеры сообщат об
успехе или неудаче



3. Производитель получит
уведомление, когда все
ведомые реплики обновятся



Пример, Scala

```
// Параметры
val servers = "localhost:9092"
val topic   = "test"

// Создаём Producer
val props = new Properties()
props.put("bootstrap.servers", servers)

val producer = new KafkaProducer(props, new LongSerializer, new StringSerializer)

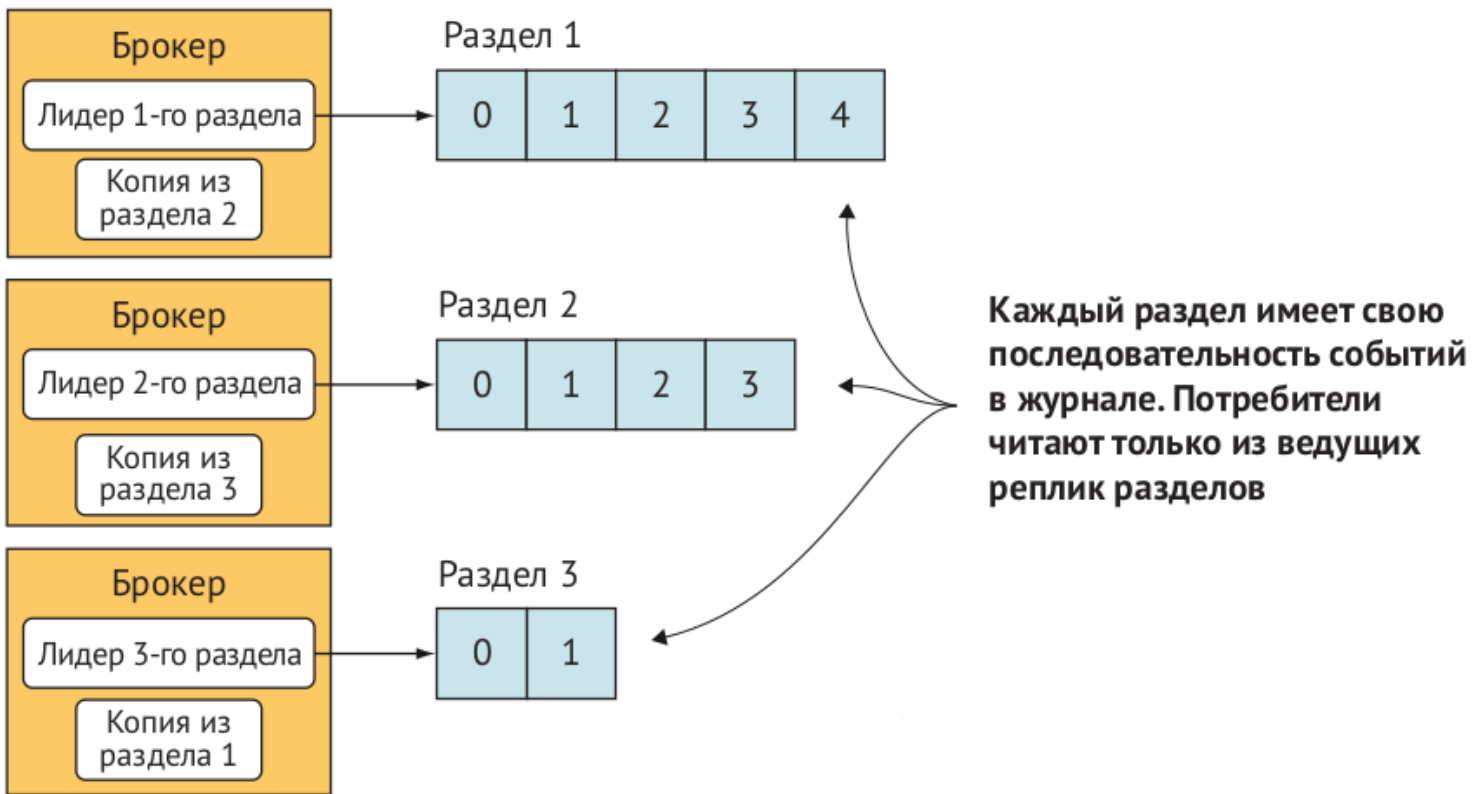
// Генерируем записи
try {
  (1 to 1000).foreach { i =>
    producer.send(new ProducerRecord(topic, i, s"Message $i"))
  }
} finally {
  producer.close()
}
```

Пример, Python

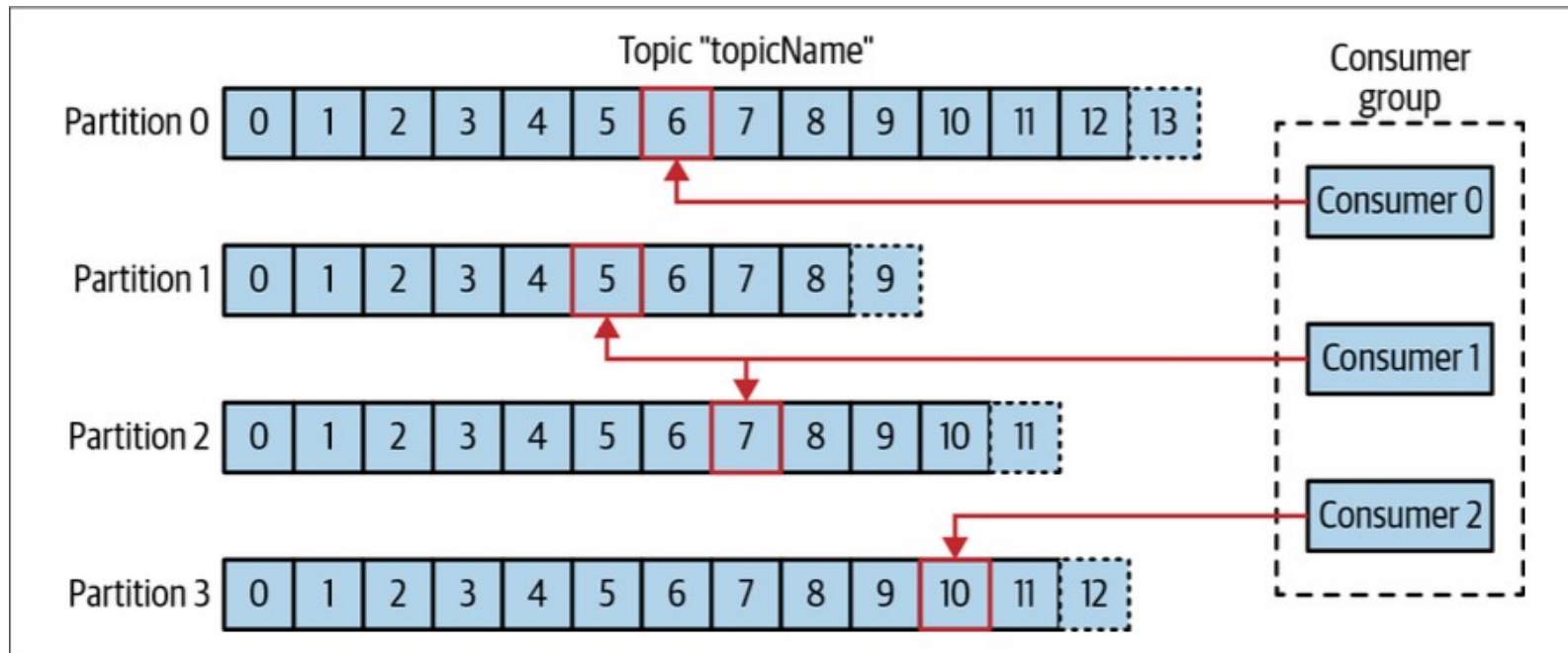
```
def produce(topic, servers):  
    producer = KafkaProducer(bootstrap_servers=servers, value_serializer=str.encode)  
  
    for i in range(1000):  
        producer.send(topic, key=i.to_bytes(length=2, byteorder='big'), value='Message '+str(i))  
  
    producer.flush()
```

Consumer

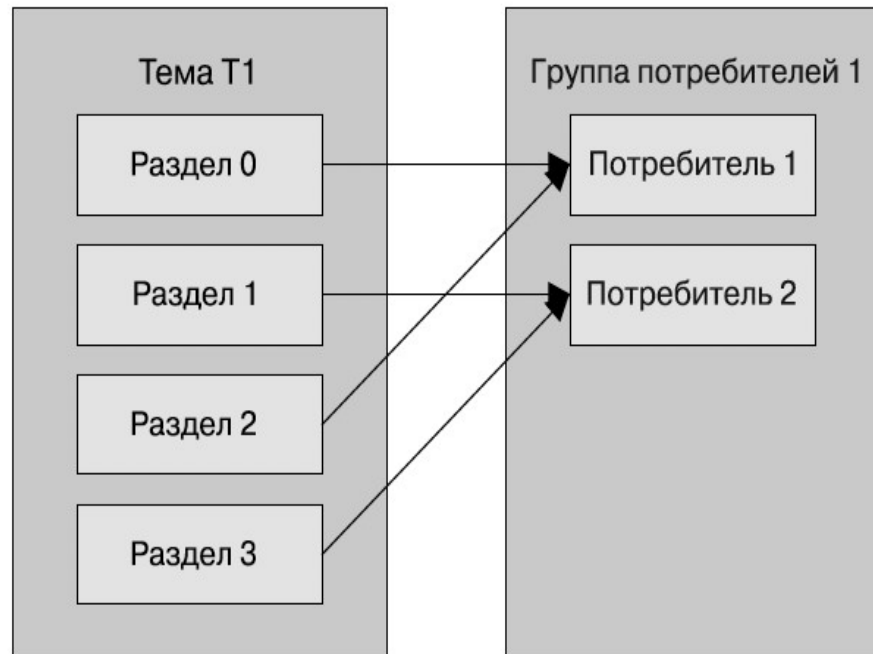
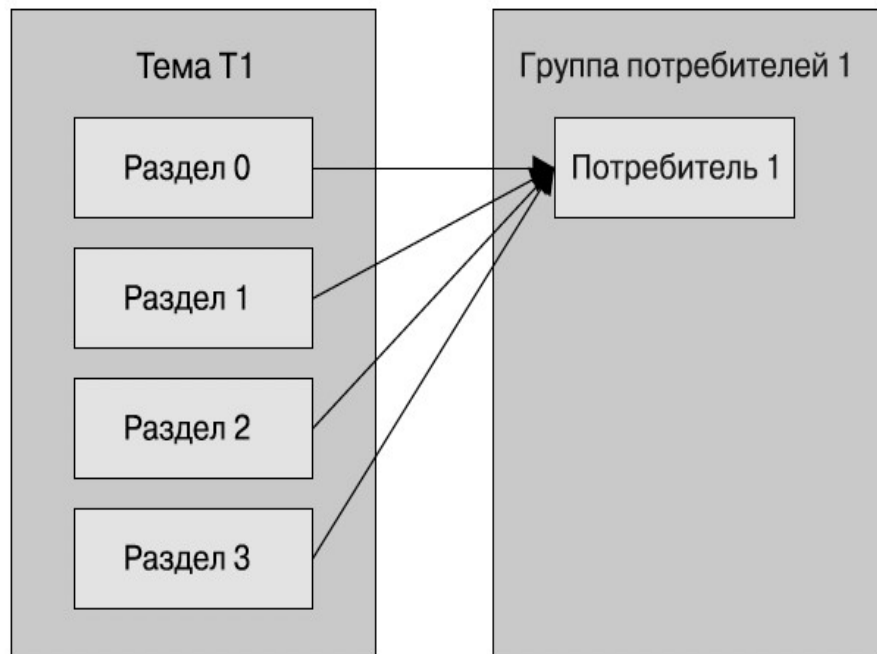
Consumer – чтение сообщений из Kafka



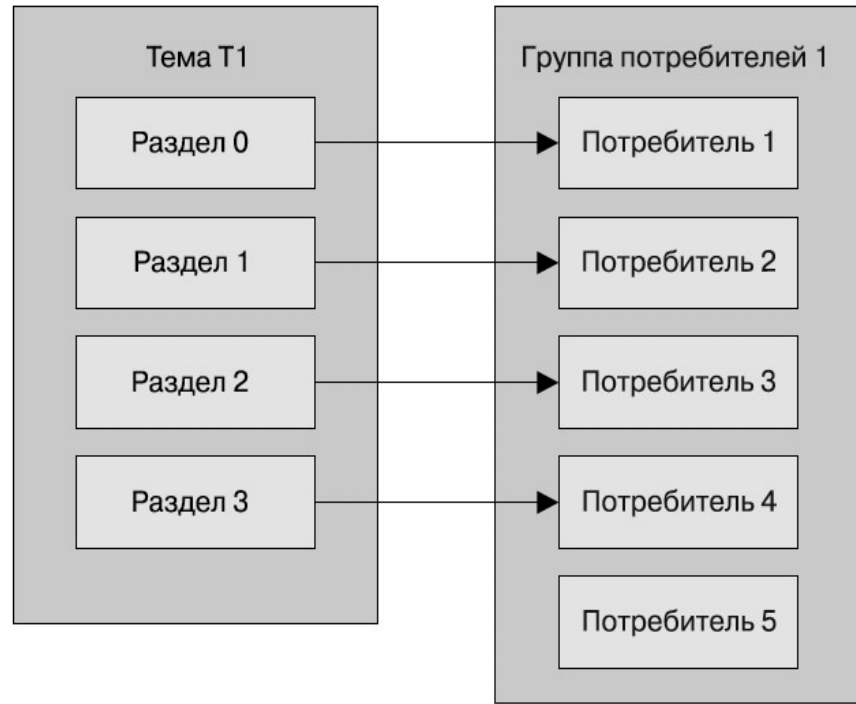
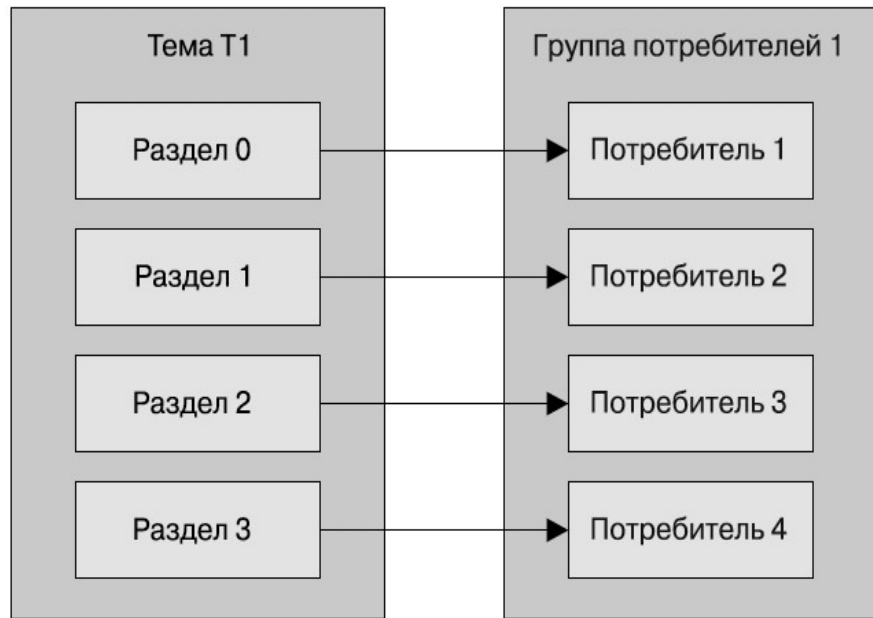
Consumer – чтение сообщений из Kafka



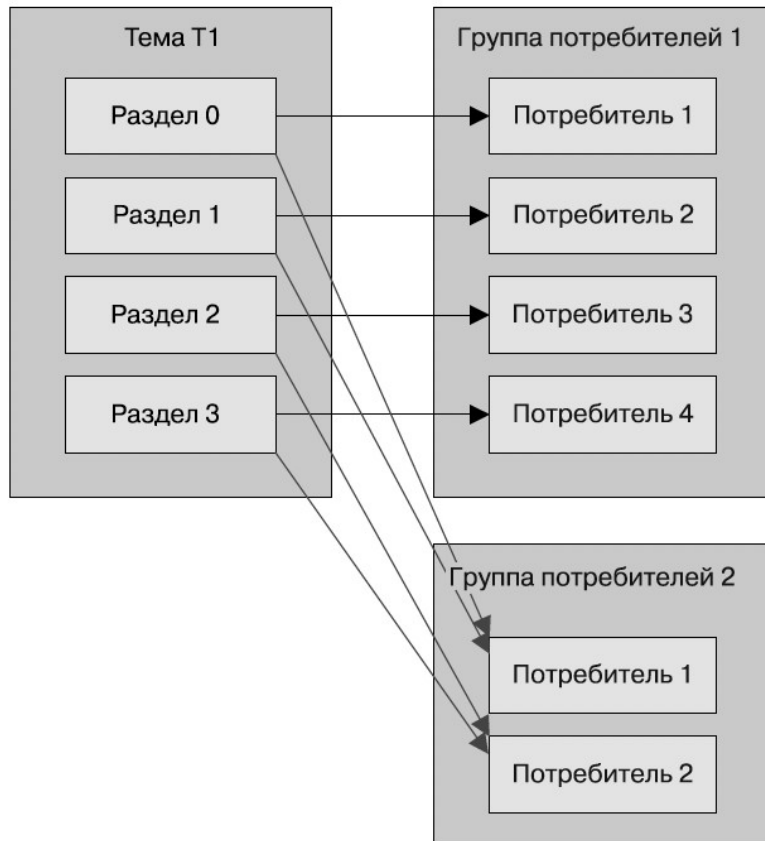
Группы потребителей



Группы потребителей



Группы потребителей



Свойства Consumer

Обязательные свойства:

- `bootstrap.servers` – список пар `host:port` брокеров
- `key.deserializer` – имя класса, используемого для десериализации ключей записей
- `value.deserializer` – имя класса, используемого для десериализации значений
- `group.id` – группа потребителей

Необязательные свойства:

- `auto.offset.reset` – поведение при начале чтения секции, если нет зафиксированного смещения:
 - `latest` – «самое позднее» (по умолчанию)
 - `earliest` – «самое раннее»
- `enable.auto.commit` – фиксировать ли смещение автоматически (`true` по умолчанию)
- `auto.commit.interval.ms` – интервал в `ms` для автоматической отправки смещений

<https://kafka.apache.org/documentation/#consumerconfigs>

Пример, Scala

```
// Параметры
val servers = "localhost:9092"
val topic   = "test"
val group   = "g1"

// Создаём Consumer и подписываемся на тему
val props = new Properties()
props.put("bootstrap.servers", servers)
props.put("group.id", group)
props.put("auto.offset.reset", "earliest")
props.put("enable.auto.commit", false)

val consumer = new KafkaConsumer(props, new LongDeserializer, new StringDeserializer)
consumer.subscribe(List(topic).asJavaCollection)
```

Подписка на темы

Подписка на одну или несколько тем:

```
consumer.subscribe(Collections.singletonList("customerCountries"));
```

Можно использовать регулярные выражения:

```
consumer.subscribe(Pattern.compile("test.*"));
```

Можно подписаться на конкретные темы и секции:

```
TopicPartition fooTopicPartition_0 = new TopicPartition("foo", 0);  
TopicPartition barTopicPartition_0 = new TopicPartition("bar", 0);  
  
consumer.assign(Arrays.asList(fooTopicPartition_0, barTopicPartition_0));
```

Цикл опроса

Цикл опроса осуществляет координацию и перебалансировку секций, отправляет контрольные сигналы и извлекает данные.

```
try {
    while (true) { ❶
        ConsumerRecords<String, String> records = consumer.poll(100); ❷
        for (ConsumerRecord<String, String> record : records) ❸
        {
            log.debug("topic = %s, partition = %d, offset = %d,
                customer = %s, country = %s\n",
                record.topic(), record.partition(), record.offset(),
                record.key(), record.value());

            int updatedCount = 1;
            if (custCountryMap.containsKey(record.value())) {
                updatedCount = custCountryMap.get(record.value()) + 1;
            }
            custCountryMap.put(record.value(), updatedCount)

            JSONObject json = new JSONObject(custCountryMap);
            System.out.println(json.toString(4)); ❹
        }
    }
} finally {
    consumer.close(); ❺
}
```

Пример, Scala

```
// Читаем тему
try {
  while (true) {
    consumer
      .poll(Duration.ofSeconds(1))
      .foreach { msg => println(s"${msg.partition}\t${msg.offset}\t${msg.key}\t${msg.value}") }
  }
} catch {
  case e: Exception =>
    println(e.getMessage)
    sys.exit(-1)
} finally {
  consumer.close()
}
```

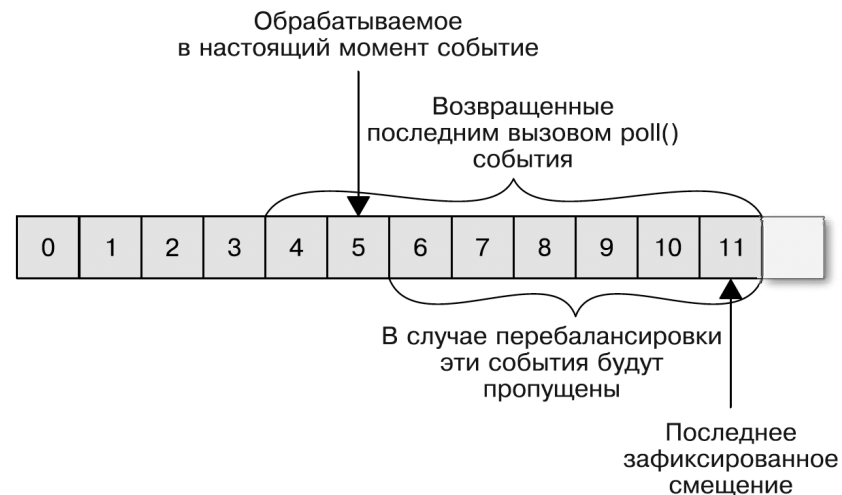


Пример, Python

```
def consume(topics, servers, group_id):  
    consumer = KafkaConsumer(bootstrap_servers=servers, group_id=group_id,  
                             value_deserializer=bytes.decode, auto_offset_reset='earliest',  
                             consumer_timeout_ms=10000)  
  
    consumer.subscribe(topics)  
  
    for message in consumer:  
        print("%d:%d: key=%s value=%s" % (message.partition, message.offset,  
                                           int.from_bytes(message.key, "big"), message.value))
```

Фиксация и смещение

- Брокер Kafka не отслеживает чтение потребителями
- Потребители могут использовать Kafka для сохранения позиции (смещения)
- *Фиксация* (commit) — действие по обновлению текущей позиции потребителя
- Потребители отправляют в специальную тему `__consumer_offsets` сообщение, содержащее смещение для каждого раздела
- Аварийный сбой потребителя или присоединение нового потребителя инициирует перебалансировку



Прослушивание на предмет перебалансировки

Работа во время смены (добавления, удаления) принадлежности секций потребителю.

Передать в `subscribe()` объект `ConsumerRebalanceListener`:

- `public void onPartitionsRevoked(Collection<TopicPartition> partitions)` – вызывается до начала перебалансировки и после завершения получения сообщения.
Здесь необходимо фиксировать смещения для следующего потребителя.
- `public void onPartitionsAssigned(Collection<TopicPartition> partitions)` – вызывается после переназначения секций потребителю, но до того, как он начнёт получать сообщения.

Прослушивание на предмет перебалансировки

Пример фиксации смещений перед сменой принадлежности секции

```
private Map<TopicPartition, OffsetAndMetadata> currentOffsets =  
    new HashMap<>();  
  
private class HandleRebalance implements ConsumerRebalanceListener { ❶  
    public void onPartitionsAssigned(Collection<TopicPartition>  
        partitions) { ❷  
    }  
  
    public void onPartitionsRevoked(Collection<TopicPartition>  
        partitions) {  
        System.out.println("Lost partitions in rebalance.  
            Committing current  
            offsets:" + currentOffsets);  
        consumer.commitSync(currentOffsets); ❸  
    }  
}  
  
try {  
    consumer.subscribe(topics, new HandleRebalance()); ❹
```



Получение записей с заданным смещением

- seekToBeginning – чтение с начала
- seekToEnd – чтение с последнего
- seek – чтение с произвольного

```
public class SaveOffsetsOnRebalance implements  
    ConsumerRebalanceListener {
```

```
    public void onPartitionsRevoked(Collection<TopicPartition>  
        partitions) {  
        commitDBTransaction(); ❶  
    }
```

```
    public void onPartitionsAssigned(Collection<TopicPartition>  
        partitions) {  
        for (TopicPartition partition: partitions)  
            consumer.seek(partition, getOffsetFromDB(partition)); ❷  
    }
```

```
    }  
  
    consumer.subscribe(topics, new SaveOffsetOnRebalance(consumer));  
    consumer.poll(0);
```

```
    for (TopicPartition partition: consumer.assignment())  
        consumer.seek(partition, getOffsetFromDB(partition)); ❸
```

```
    while (true) {  
        ConsumerRecords<String, String> records =  
            consumer.poll(100);  
        for (ConsumerRecord<String, String> record : records)  
        {  
            processRecord(record);  
            storeRecordInDB(record);  
            storeOffsetInDB(record.topic(), record.partition(),  
                record.offset()); ❹  
        }  
        commitDBTransaction();  
    }
```

Литература

Список литературы

- Apache Kafka. Потокковая обработка и анализ данных, 2-е изд — <https://www.piter.com/collection/all/product/apache-kafka-potokovaya-obrabotka-i-analiz-dannyh-2-e-izdanie>
- Kafka в действии — <https://dmkpress.com/catalog/computer/data/978-5-93700-118-4>
- Kafka Streams в действии — https://www.piter.com/product_by_id/132649774
- Kafka Streams и ksqlDB: данные в реальном времени — <https://www.piter.com/collection/new/product/kafka-streams-i-ksqldb-dannye-v-realnom-vremeni>
- Потокковая обработка данных — <https://dmkpress.com/catalog/computer/data/978-5-97060-606-3>
- Проектирование событийно-ориентированных систем — <https://dmkpress.com/catalog/computer/os/978-5-6042412-1-9>
- Создание событийно-управляемых микросервисов <https://bhv.ru/product/sozdanie-sobytiijno-upravlyaemyh-mikroservisov>

Ссылки

- Apache Kafka – <http://kafka.apache.org/documentation>
- Confluent Platform – <https://www.confluent.io/product/confluent-platform>
- Quick Start for Confluent Platform – <https://docs.confluent.io/platform/current/platform-quickstart.html>
- Confluent Developer – <https://developer.confluent.io>
- ksqlDB – <https://docs.ksqldb.io/en/latest>
- Kafka Streams – <https://kafka.apache.org/documentation/streams>
- Kafka Python client – <https://github.com/dpkp/kafka-python>
- Confluent's Python Client for Apache Kafka – <https://github.com/confluentinc/confluent-kafka-python>
- Spring Cloud Stream – <https://spring.io/projects/spring-cloud-stream>



Рефлексия

Рефлексия



С какими впечатлениями уходите с вебинара?



Как будете применять на практике то, что узнали на вебинаре?

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Следующий вебинар



Apache NiFi



Ссылка на вебинар
будет в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной
лентой

Спасибо за внимание!

Приходите на следующие вебинары



Заигрин Вадим

Ведущий эксперт по технологиям, Сбербанк

vzaigrin@yandex.ru

<https://t.me/vzaigrin>

